

Repetition penalties are not a well-defined intervention: gauge non-invariance and a closed-form corruption threshold in logit-space repetition penalties

Peter Hollows

Abstract

The repetition penalty is a standard decoding knob: one scalar θ , set to around 1.1–1.3 to suppress repetition and assumed to mean the same thing from one model to the next. As implemented in HuggingFace Transformers (the CTRL divide-if-positive / multiply-if-negative rule on *raw* logits, inherited in identical form by vLLM and llama.cpp), it is not the clean, portable knob it is taken to be, for one structural reason. The sign-branch is the community’s own fix for an old bug, in which the naive divide raised the probability of negative-logit tokens; but the softmax is shift-invariant, so a model’s logit zero-point is left unconstrained by the training objective, and branching on the sign reads exactly that unconstrained coordinate. The accepted fix is thus itself ill-posed, with two consequences. Within a single model, the same θ is not a single intervention: a behaviorally invisible shift that changes nothing at $\theta = 1$ flips 50–94% of greedy tokens at $\theta = 1.3$ (five models up to 7B, base and RLHF), and across models a fixed θ does not transfer, since real checkpoints sit on different sides of the sign boundary. Where the penalty acts, it deterministically swaps the model’s top token for the runner-up at a closed-form threshold, $g < z_{\text{top}}(1 - 1/\theta)$, which at a routine $\theta = 1.3$ drops schema-valid JSON output to 0%. Both effects replicate inside vLLM and llama.cpp through each stack’s own sampler. Both vanish if the penalty is applied to normalized log-probabilities, an operator HuggingFace already ships but leaves off by default and applies after the penalty. The fix requires no new machinery; the contribution is the diagnosis.

1 Introduction

Open-ended neural text generation is prone to degeneration: greedy and low-temperature decoding fall into repetitive loops in which the model re-emits the same token or phrase indefinitely [5, 22]. A standard, cheap countermeasure is the *repetition penalty*: before sampling, the logits of tokens that have already appeared in the context are pushed down, so that recurrence is discouraged. Introduced in CTRL [8], the *multiplicative* (sign-branched divide/multiply) form of the penalty is shipped as `repetition_penalty` in HuggingFace Transformers [24] and is carried, in the same form, into vLLM and llama.cpp’s `repeat_penalty`; a separate *subtractive* family (presence/frequency penalties, exposed by the major hosted APIs and also by vLLM) behaves differently and, as we note in Section 7 (Table 5), is immune to the first effect we identify. Our results concern the multiplicative CTRL form. Throughout, our main experiments use the HuggingFace implementation; both effects are additionally replicated inside vLLM and llama.cpp through each stack’s own sampler (Section 6). Practitioners reach for it reflexively, set it to a small value (commonly 1.1–1.3), and reason about it as a clean monotone dial: larger θ means “more anti-repetition,” the setting is assumed comparable across models, and the intervention is assumed harmless to everything that is not a degenerate loop.

This paper shows that, for the penalty as it is actually implemented, all three of those assumptions fail in measurable ways. A `repetition_penalty` value tuned on one model is not the same operation on another, so it does not transfer in any principled sense; and at a routine $\theta = 1.3$ the operator can deterministically corrupt structured output, dropping a model’s rate of valid, schema-conformant JSON to zero. Both effects, and the fix for both, trace to a single structural decision, itself introduced as a fix. CTRL’s original penalty divided every seen logit by θ , but dividing a *negative* logit by $\theta > 1$ moves it toward zero and *raises* the token’s probability, so the penalty backfires on exactly the tokens it should suppress; this was reported to HuggingFace in 2019 [21]. The remedy, now standard, is to branch on the *sign* of each raw logit (multiply negatives, divide positives), so a seen token’s probability always falls. That remedy works at the level it was aimed at. Our observation is one level up: branching on a raw logit’s sign reads a quantity the model is free to choose arbitrarily, because the softmax is invariant to a global additive shift of the logits. The accepted fix for the naive penalty is itself ill-posed. We make this precise and draw out two consequences.

A1: the penalty is not gauge-invariant (Section 3). The simplest form of the result lives *inside a single model*. Adding a constant c to every logit is a provable no-op at $\theta = 1$ (we confirm it is token-for-token invisible under both greedy and seeded sampling); yet with the penalty on, two such provably-equivalent gauges of the *same* model disagree on 50–94% of greedy tokens at $\theta = 1.3$. The two gauges differ only by an additive constant that has no effect on the output distribution. The cross-model reading follows as a corollary: because a model’s logit zero-point is never trained or constrained, real checkpoints sit at different points on the sign boundary, so a fixed θ is a different operation on each. (Multiplicative penalties on un-normalized logits would fail to transfer across models for mundane reasons too, since logit scales and calibration differ; the within-model result is the one that isolates the gauge, rather than scale, as the cause.)

A2: where it acts, it is a closed-form corruption threshold (Section 4). Structured output (JSON, source code, quoted strings, list markers) contains *grammar-obligatory* repetition: the same delimiters (`{`, `}`, `[`, `]`, `,`, `"`, indentation) must recur, and the penalty cannot tell “the grammar requires this token” from “the model is stuck in a loop.” We derive the exact condition under which the penalty flips a confident, correct token to the runner-up— $g < z_{\text{top}}(1 - 1/\theta)$ —and show it predicts the realized flip with balanced accuracy ≥ 0.999 on two independently trained 7B code models, with every flip landing on the pre-penalty second choice.

We deliberately separate what is verified from what is not. The original framing of A2 predicted that corruption would strike the model’s *most confident* tokens first and would be sharply *code-specific*; both are refuted by our data (Section 4.3). Reporting these negative sub-results is part of the contribution: the surviving statement (a gap-driven threshold that corrupts all token types similarly but lands disproportionately on the delimiters that structured output cannot avoid) is more precise and better supported than the original.

Both results concern the same operator, and both are removed by the same one-line change: apply the penalty to normalized log-probabilities rather than raw logits (Section 5). Normalization is itself shift-invariant, so it erases the gauge c ; and it makes every value non-positive, so the sign-branch stops being a branch at all. We confirm this directly: under the normalized operator the A1 flip rate collapses to 0.000 at every θ on all three models, and the A2 structural corruption rate drops by roughly $27\times$. HuggingFace already ships this normalization operator but leaves it disabled by default and, even when enabled, runs it *after* the penalty, too late to help. The remediation is a reorder of existing machinery; the observation is that it is not the default.

2 The operator under test

We study the operator exactly as implemented in HuggingFace Transformers (`RepetitionPenaltyLogitsProcessor`), which is the de facto reference and matches the CTRL formulation [8]. Let $z \in \mathbb{R}^V$ be the logits the model emits for the next token over a vocabulary of size V , and let $S \subseteq \{1, \dots, V\}$ be the set of token ids that have already appeared in the context (prompt plus generated text). The penalty with strength $\theta \geq 1$ transforms only the seen tokens, by branching on the sign of each logit:

$$z'_i = \begin{cases} z_i/\theta & i \in S \text{ and } z_i \geq 0, \\ z_i \cdot \theta & i \in S \text{ and } z_i < 0, \\ z_i & i \notin S. \end{cases} \quad (1)$$

The next token is then drawn from $\text{softmax}(z')$ (or chosen as $\arg \max z'$ under greedy decoding). The verbatim implementation is:

```
score = torch.gather(scores, 1, input_ids)
# if score < 0 multiply by penalty (more negative); else divide (smaller positive)
score = torch.where(score < 0, score * self.penalty, score / self.penalty)
scores_processed = scores.scatter(1, input_ids, score)
```

Two implementation facts matter for everything that follows. First, the operator acts on *raw* logits: there is no normalization before it. Second, HuggingFace does ship a normalization processor,

```
class LogitNormalization(LogitsProcessor):
    def __call__(self, input_ids, scores):
        return scores.log_softmax(dim=-1)
```

but `GenerationConfig.renormalize_logits` defaults to off, and when it is enabled the processor is appended to the *end* of the processor list, after the repetition penalty. So in the default configuration, and even in the non-default “renormalize” configuration, the penalty sees un-normalized logits. This is precisely the degree of freedom we exploit in Section 3 and remove in Section 5.

The sign-branch is not incidental: it is the community’s fix for the naive divide (which raised the probability of negative-logit tokens; Section 1, 21). What matters here is that the two branches meet at $z_i = 0$ with *different slopes*, so the operator’s behavior turns on which side of $z_i = 0$ each seen logit lands, and $z_i = 0$ is a point the softmax lets the model place anywhere (Section 3).

3 A1: gauge non-invariance

3.1 Theory

The gauge. For any constant $c \in \mathbb{R}$ and the all-ones vector $\mathbf{1}$,

$$\text{softmax}(z + c\mathbf{1})_i = \frac{e^{z_i+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \text{softmax}(z)_i. \quad (2)$$

The output distribution is invariant to a global additive shift of the logits. Adding c to every logit is identical to adding c to every component of the final linear layer’s bias (`lm_head.bias += c`); the greedy choice is therefore unchanged exactly, and sampled output is unchanged token-for-token under a fixed random seed. The shift c is a pure *gauge*: c parameterizes an orbit of logit vectors that the softmax collapses to one and the same distribution, so any well-defined function of the model’s *behavior* must be invariant to it. We use “gauge” in exactly this sense (an unobservable

redundancy a well-posed decoding operator ought to ignore), and nothing in the argument relies on the term beyond it. At $\theta = 1$ the penalty is the identity and this invisibility is exact.

The sign-branch breaks it. The penalty in Eq. (1) is not a function of the distribution; it is a function of the raw logits, and specifically of their signs. Apply the gauge first. A previously-seen token with logit x now has logit $x + c$, and the penalty subtracts from it

$$\Delta(x; c) = \begin{cases} (x + c) \left(1 - \frac{1}{\theta}\right) & x + c \geq 0 \quad (\text{divide branch}), \\ |x + c| (\theta - 1) & x + c < 0 \quad (\text{multiply branch}). \end{cases} \quad (3)$$

Every *competing* (unseen) token also shifts by c , so the constant c cancels in all the comparisons that decide the next token, *except* through the nonlinear, sign-dependent transform applied to the penalized tokens. The drop Δ is not shift-invariant: it depends on where $x + c$ sits relative to the kink at zero, which the gauge c controls.

Concretely, consider a high-probability repeated token. With $c = +5$ it sits deep in the positive/divide regime, where Δ grows with $x + c$, and it is heavily penalized. With $c = -5$ the same token can be pushed just below zero into the multiply branch, where $|x + c|$ is small and $\Delta \approx 0$: the penalty nearly vanishes on the token it was meant to suppress. Same model, same θ , same prompt; two gauges that are provably indistinguishable at $\theta = 1$ now implement two different interventions.

3.2 Design

We implement the gauge as a logits transform $z += c$ applied immediately before the penalty in a hand-written decode loop. This is mathematically identical to shifting the `lm_head` bias, but makes the ordering explicit and sidesteps GPT-2’s tied, bias-free output head. The penalty uses the exact HuggingFace semantics of Eq. (1) over all previously-seen ids (prompt and generation). We run three independently trained models (`gpt2`, `gpt2-large` [15], and `pythia-2.8b` [1]) in fp32, over a grid of $\theta \in \{1.0, 1.02, 1.05, 1.1, 1.15, 1.3\}$ and $c \in \{-5, -3, -1, 0, 1, 3, 5\}$, with 16 fixed prompts and 200 generated tokens each.

No-op control. The validity gate is that the gauge must be invisible when the penalty is off. At $\theta = 1$ we require the generated sequence to be token-for-token identical across all c , for every prompt, under both greedy and seeded sampling. Greedy invariance is trivial; the seeded-sampling leg (a shared RNG stream across gauges) is the stronger check, since it probes the full sampled distribution rather than only its argmax. We do not enforce bitwise equality in the harness; we *verify* softmax shift-invariance empirically, and it holds: this control passed exactly, 32/32 (prompt, mode) pairs identical, with 0 of 3200 greedy tokens differing between $c = +5$ and $c = -5$ at $\theta = 1$. Because the gauge is added inside the same decode loop that runs every $\theta > 1$ condition, dtype, RNG, and code path are differenced out within each comparison; any divergence at $\theta > 1$ is therefore caused by the gauge $\times$ sign-branch interaction, not by numerics or a bug.

Endpoint. The quantity that directly operationalizes the claim “a provable no-op changes generation” is the per-step argmax *flip rate*: the fraction of greedy decoding steps at which the $c = +5$ and $c = -5$ runs (both provable no-ops at $\theta = 1$) choose different tokens. Unlike a repetition ratio, the flip rate cannot saturate: it is 0 exactly when behavior is unchanged and rises as the gauge bites. We report it as a function of θ , with a per-prompt bootstrap 95% confidence interval (10^4 resamples) at $\theta = 1.3$.

Table 1: A1: argmax flip rate between the $c = +5$ and $c = -5$ gauges (both provable no-ops at $\theta = 1$). The gate column confirms the no-op is exact; the $\theta = 1.3$ column shows the same provably-equivalent shift selects different greedy output once the penalty is on. The effect replicates at 7B and is undiminished by RLHF (last two rows).

Model	flip rate ($\theta=1$)	flip rate ($\theta=1.3$)	95% CI
gpt2-large	0.000	0.941	[0.916, 0.959]
pythia-2.8b	0.000	0.905	[0.876, 0.933]
gpt2	0.000	0.497	[0.333, 0.655]
Qwen2.5-7B (base)	0.000	0.871	[0.832, 0.906]
Qwen2.5-7B-Instruct	0.000	0.897	[0.864, 0.927]

3.3 Results

The flip rates are in Table 1. At $\theta = 1$ every model flips exactly 0% of tokens: the no-op control passes exactly. At $\theta = 1.3$ the *same model*, run under two provably-equivalent gauges ($c = +5$ and $c = -5$), differs on 50% (gpt2), 91% (pythia-2.8b), and 94% (gpt2-large) of greedy tokens, bootstrap intervals far above zero throughout; the model itself is unchanged. The effect is not confined to small base models: it replicates at 7B and is undiminished by post-training, with the no-op gate exactly 0 and flip rates of 0.871 on Qwen2.5-7B and 0.897 on its instruction-tuned/RLHF variant Qwen2.5-7B-Instruct (Table 1), if anything stronger than on the small models. The synthetic shift $c = \pm 5$ used here is a controlled sweep (it maps the response surface and pins the no-op gate), and the constant 5 is arbitrary. Below we use a gauge that involves no arbitrary constant: each model’s *own median* logit, a re-centring it could equally have shipped, which flips more tokens (Table 2). The flip rate rises from zero as soon as the penalty engages (0.61–0.78 at $\theta = 1.02$); its magnitude is non-monotone in θ for the small loop-prone gpt2, and the effect manifests as changed content rather than changed repetition rate (audit in App. A).

Real checkpoints differ in zero-point. Nothing about the gauge c requires a synthetic shift. A model’s logit zero-point is never an optimization target and is not constrained by any loss term; it is determined incidentally by initialization, the (often absent or untied) output bias, weight decay, and training dynamics, and so it differs across checkpoints and architectures. To quantify this, we measured where each of our five real checkpoints sits relative to the sign boundary the penalty branches on, at decode time (Table 2). The spread is wide: the fraction of already-seen tokens whose logit is positive (i.e. which the penalty *divides* rather than *multiplies*) ranges from 0.09 on gpt2 to 0.99 on Qwen2.5-Coder-7B. At one and the same `repetition_penalty=1.3`, then, the operator divides the logits of $\sim 9\%$ of seen tokens on gpt2 (so it mostly *multiplies*) and $\sim 99\%$ on Qwen2.5-Coder (so it almost always *divides*): on these two checkpoints the sign-branch sends the penalty down opposite paths, with no synthetic shift applied to either. gpt2 is the extreme case (a small model with a tied, bias-free head and a median top-1 logit of -97.5 , far below the others), but the effect is not an artifact of that one outlier: even setting gpt2 aside, the divide-fraction still spans 0.75 to 0.99 across the four remaining models, a wide enough range that a fixed θ applies a materially different penalty to each.

The natural offsets are large. Re-centring each model’s logits by its own median (a behaviorally invisible gauge it could equally have been trained into) and comparing the resulting greedy trajectory to the raw one (both provable no-ops at $\theta = 1$) flips 57–86% of tokens per model at $\theta = 1.3$ (76.4% pooled across the five), *more* than a synthetic $c = 5$ shift (31.6% pooled). The gauge non-invariance

Table 2: Where five real checkpoints sit relative to the penalty’s sign boundary, measured at decode time ($c = 0$, $\theta = 1$). The fraction of seen-token logits that are positive (the divide branch) spans 0.09–0.99 (0.75–0.99 excluding the small gpt2 outlier), so a fixed θ is a materially different operation on each. The last column is the per-model A1 flip rate between the model’s raw gauge and its own median-centred gauge (a gauge it could equally have shipped) at the *natural* offset magnitude.

Model	frac. seen logit > 0 (divide)	median top-1 logit	flip @ natural $\theta=1.3$
gpt2	0.091	−97.5	0.569
gpt2-large	0.746	11.9	0.781
starcoder2-7b	0.828	15.5	0.858
pythia-2.8b	0.938	17.6	0.815
Qwen2.5-Coder-7b	0.986	24.2	0.796
<i>pooled across the five checkpoints</i>			0.764

is therefore not an artifact of an adversarially large injected shift; it is at least as strong at the offset magnitudes real checkpoints already exhibit. Consequently `repetition_penalty=1.3` already denotes different interventions on different real models, and a setting tuned on one does not transfer to another in any principled sense, because the two models do not share the coordinate system the penalty operates in.

4 A2: the corruption threshold

4.1 The closed form

Structured output contains repetition that the grammar *requires*: a JSON object must close every `{` with a `}`, separate items with `,`, and quote keys with `"`; Python must re-emit indentation and colons. These delimiters have, by construction, appeared before, so they are in the penalized set S , and they are typically the model’s confident top choice at the position where the grammar demands them. The penalty does not distinguish grammar-obligatory recurrence from degenerate looping, so it penalizes them.

Consider such a position. Let the correct token be the pre-penalty top choice with logit $z_{\text{top}} > 0$ (so it is in the divide branch), and let the runner-up be a gap $g = z_{\text{top}} - z_2 \geq 0$ below it. Assume the runner-up is unpenalized (it is not a delimiter that just occurred). The penalty maps $z_{\text{top}} \mapsto z_{\text{top}}/\theta$ but leaves the runner-up at $z_2 = z_{\text{top}} - g$. The emitted (greedy) token flips from the correct top choice to the runner-up exactly when

$$\frac{z_{\text{top}}}{\theta} < z_{\text{top}} - g \iff g < z_{\text{top}} \left(1 - \frac{1}{\theta}\right). \quad (4)$$

The right-hand side is the *vulnerable gap*: any correct token whose lead over the runner-up is smaller than $z_{\text{top}}(1 - 1/\theta)$ is deterministically corrupted. The threshold is exact (not probabilistic) under greedy decoding, and it is a hard threshold in θ : the flip occurs discontinuously, not gradually.

4.2 Design

We test Eq. (4) on `bigcode/starcoder2-7b` [11], a model whose domain makes grammar-obligatory delimiters abundant. Decoding is greedy with $\theta \in \{1.0, 1.1, 1.2, 1.3, 1.4, 1.5\}$ over 18 fixed prompts (6 JSON, 6 Python, and 6 prose as a control with no grammar-obligatory repetition), 256 generated

tokens each. The model runs in bf16, but at every step we upcast the final logits to fp32 before the penalty, the argmax, and the threshold evaluation, so that the formula test is not blurred by bf16 rounding. At each position we record the pre-penalty top-1 id and z_{top} , the runner-up id and z_2 , the gap g , indicator flags (`top1_seen`, `top1_positive`, `top2_seen`, `top1_structural`), and the post-penalty emitted id, from which we derive `flip` and `emitted_is_runnerup`. A token is tagged *structural* if its decoded form is made entirely of delimiter characters `{ } [ ] ( ) , ; =` or whitespace/indentation.

Validity gate and a numerical subtlety. At $\theta = 1.0$ the penalty is the identity, so the flip rate must be exactly 0 at every position; this passes. Reaching it required one fix worth flagging for replicators: a naive bf16 `topk`-versus-`argmax` comparison reports spurious “flips” at ties, because the two can break an exact tie differently. The harness therefore defines the pre-penalty top-1 by the same `argmax` used post-penalty; with that, the $\theta = 1.0$ gate is clean.

4.3 What is verified

We evaluate Eq. (4) on the *clean* positions where it is exact: the top-1 token is seen and positive (divide branch) and the runner-up is unseen (unpenalized). Pooling over $\theta \in \{1.1, \dots, 1.5\}$ on the code prompts gives $n = 3973$ clean positions (25.9% of all penalized code positions; the remaining 74.1% are the two-sided case where the runner-up is itself penalized, treated below), of which 2225 actually flip.

On this slice the threshold is, by derivation, an algebraic identity for greedy argmax: the penalty maps $z_{\text{top}} \rightarrow z_{\text{top}}/\theta$ and leaves the runner-up untouched, so $g < z_{\text{top}}(1 - 1/\theta)$ is the flip condition. We therefore do not read the predicate accuracy (balanced accuracy 0.999; TPR 1.000, TNR 0.999; the only departures from 1.000 are bf16 rounding) as the empirical content. The empirical content is threefold and is *not* guaranteed by the algebra:

- 100% of the 2225 flips emit the pre-penalty runner-up. When the penalty corrupts a token, it does not scatter probability diffusely; it deterministically promotes the specific second-place token. A diffuse-corruption alternative would not produce this.
- The $\theta = 1$ gate holds (0 flips), so every flip is penalty-induced, not an artifact.
- The threshold is crossed at scale: a penalty of $\theta \in [1.1, 1.5]$ flips $\sim 31\%$ of all greedy code tokens (not only clean ones), so real structured decoding lands in this regime routinely.

Where the penalty acts, it either leaves the choice alone or deterministically replaces the model’s top token with its runner-up, at a point computable in closed form from the pre-penalty logits. The two-sided positions (74.1%) obey a more general two-sided inequality (both top-1 and runner-up penalized) that we state in Section 8 but do not separately tabulate; the exact closed form above is the one-sided regime.

Replication on a second code model. To check that the threshold is a property of the operator rather than of one checkpoint’s logit geometry, we re-ran the identical harness on **Qwen2.5-Coder-7B** [7], an independently trained code model. The result is, if anything, sharper: on $n = 3584$ clean penalized positions (2484 flips) the closed form achieves balanced accuracy 1.000 (TPR 1.000, TNR 1.000), 100% of flips emit the pre-penalty runner-up, and the $\theta = 1.0$ validity gate passes. The same gap-driven signature recurs (Section 4.3): flipped tokens have lower mean z_{top} (23.9) than non-flipped (28.8). The corruption threshold is thus demonstrated on two independent 7B code models.

What is refuted. We pre-registered two stronger claims about the threshold, and our own data refute both. It does *not* corrupt the most confident tokens first: corruption is *gap-driven*, so flipped tokens have *lower* mean z_{top} than non-flipped ones (e.g. 17.5 vs 22.1 on StarCoder2, and the same ordering on Qwen2.5-Coder). And it is not sharply code-specific: the code-vs-prose structural-flip ratio is real but model-dependent ($2.3\times$ on StarCoder2, $5.6\times$ on Qwen2.5-Coder), and prose is corrupted at a comparable overall per-token rate. The penalty corrupts all token types at a similar rate and lands disproportionately on delimiters only because structured output exposes more of them. The full audit, with CIs, is in Appendix A. What survives is the narrower statement: the penalty deterministically flips the greedy token to the runner-up exactly when $g < z_{\text{top}}(1 - 1/\theta)$: a gap-driven threshold that corrupts all token types similarly and so lands disproportionately on the grammar-obligatory delimiters structured output cannot avoid.

5 The fix: normalize before penalizing

Both effects trace to one decision (branching on the sign of an un-normalized logit), and both are removed by one change: apply the penalty to normalized log-probabilities, $\ell = \log \text{softmax}(z)$, instead of to raw logits z . This is less “remove a degree of freedom” than “use the right coordinate.” Among all the gauges $z + c\mathbf{1}$ that the softmax identifies, ℓ is the canonical representative, the one in which the model’s own distribution is actually written ($\sum_i e^{\ell_i} = 1$); the operator should read there rather than at a zero-point the training objective leaves unconstrained. Two properties of ℓ do the work.

(a) Normalization erases the gauge. $\log \text{softmax}$ is itself shift-invariant: $\log \text{softmax}(z + c\mathbf{1}) = \log \text{softmax}(z)$, since the additive c cancels between the logit and the log-partition. Penalizing ℓ therefore cannot depend on c , and the A1 divergence is eliminated by construction: the operator becomes invariant to the coordinate choice it currently reads.

(b) The penalty becomes tempering, with a positive identity. Every log-probability satisfies $\ell_i \leq 0$, so under Eq. (1) every penalized token takes the *multiply* branch, $\ell_i \mapsto \theta \ell_i$, moving each log-probability uniformly further from zero. There is no kink whose location depends on an arbitrary zero-point, and no token can sit on the “cheap” side of a sign boundary the way the A1 mechanism exploits. And the operation now has a name: $\theta \ell_i = \theta \log p_i = \log p_i^\theta$, so in probability space it is $p_i \mapsto p_i^\theta$ (before renormalization): the fixed operator tempers each seen token’s probability toward zero by a common power. This is a monotone, well-defined suppression measured in log-probability units, a coordinate-free quantity, and hence at least *dimensionally* comparable across models (we do not claim a fixed θ is numerically optimal across models, only that it denotes the same kind of operation). The normalized penalty does act on log-probabilities ($O(1)$ in magnitude) rather than raw logits ($O(10)$), so the same θ is a gentler push, which is exactly why A2 corruption collapses below.

Remediation. The remediation is a design principle, not a patch to one library: a decoding-time penalty should act on a shift-invariant quantity (normalized log-probabilities, or any transform from which the gauge has been removed), and the normalization must run before the penalty, not after. For the operator at hand, that means penalizing $\log \text{softmax}(z)$ rather than z . It applies unchanged to every stack carrying the multiplicative CTRL form (HuggingFace, vLLM, llama.cpp; Table 5), since they share the operator, and it is nearly free: no new machinery, only a reorder.

Table 3: The fix (apply the penalty to $\log \text{softmax}(z)$). A1 gauge divergence is eliminated on all three models; A2 delimiter corruption is reduced $\sim 27\times$ and the threshold formula ceases to fire.

	metric	raw logits (default)	normalized (fix)
A1: flip rate at $\theta=1.3$	gpt2-large	0.941	0.000
	pythia-2.8b	0.905	0.000
	gpt2	0.497	0.000
A2: StarCoder2-7B	code structural-flip rate	0.138	0.005
	threshold balanced accuracy	0.999	0.794

HuggingFace makes the point concrete: it already ships exactly this normalization as `LogitNormalization(scores.log_softmax(dim=-1))`, but off by default (`renormalize_logits=None`) and, when enabled, appended *after* the penalty, so it normalizes scores the penalty has already modified.¹ The fix is thus reachable in stock machinery yet absent from the default path, which is the finding. Its one cost is that, being gentler per unit θ , the normalized penalty does not inherit existing tuned values: θ must be re-set; in exchange, the same θ denotes the same operation on every model.

Why fix it rather than abandon it? One could instead switch to a shift-invariant subtractive penalty (Table 5) or a purpose-built alternative such as the LZ penalty [4]; those are complementary. We focus on the in-place, one-line correction because the multiplicative CTRL form is the shipped default and existing tuned presets are written against it.

Demonstration. We re-ran both experiments with a single change, inserting $z \leftarrow \log \text{softmax}(z)$ immediately before the penalty (exposed as a `-fix` flag) and holding everything else identical, and the prediction holds (Table 3). For A1, the gauge-induced flip rate, which the un-normalized operator drives to 0.50–0.94 at $\theta = 1.3$, collapses to 0.000 at every θ on all three models: the gauge c becomes invisible everywhere, not just at $\theta = 1$, exactly as shift-invariance requires. For A2, the code structural-flip rate falls from 0.138 to 0.005 ($\sim 27\times$ lower), and the closed-form threshold of Eq. (4) stops being predictive (balanced accuracy $0.999 \rightarrow 0.794$), the intended outcome: there is no longer a sign-branch threshold for it to predict.²

The reason corruption nearly vanishes (rather than the penalty being switched off outright) is instructive. After normalization a confident correct delimiter has log-probability close to 0, so the multiply branch ($\ell_i \mapsto \ell_i \cdot \theta$) barely moves it; the penalty’s effect is now proportional to how *improbable* a token already is, so it pushes down overused tokens while leaving confident, grammar-required ones essentially untouched. This is the intended behavior of a repetition penalty; the un-normalized sign-branch does not provide it.

The corruption matters end-to-end, and the fix recovers it. The results so far are mechanistic (per-token flips, threshold accuracy). To confirm the per-token corruption changes something

¹The related idea of normalizing *after* the logits processors is known in a different setting: because processors can leave beam scores un-normalized, HuggingFace re-normalizes them (via `log_softmax`) so beam-search comparison is fair [6]. Applying normalization *before* the penalty, and to the repetition penalty specifically, is the point here.

²The residual 0.794 is asymmetric (TPR 1.000, TNR 0.587): under the normalized operator the multiply branch sends nearly every penalized log-probability across the predicate’s right-hand side, so $g < z_{\text{top}}(1 - 1/\theta)$ (still evaluated on the raw-logit z_{top} and g , and no longer the condition governing the flip) fires almost everywhere while actual flips have collapsed. The formula stops *discriminating* flips because there are almost none left to discriminate; it is not a competing threshold.

Table 4: End-to-end quality under the raw vs. normalized penalty (single model, Qwen2.5-Coder-7B, greedy, complete generations; JSON $n = 48$ over six schemas). At a routine $\theta = 1.3$ the raw operator yields no valid JSON; the fix preserves validity. For HumanEval only the relative θ -trend is meaningful (the absolute baseline is depressed by our extraction harness; see text), so we report it as corroborating, with JSON conformance as the clean headline. This is an existence demonstration of end-to-end impact, not a universal rate.

Metric	operator	$\theta=1.0$	$\theta=1.1$	$\theta=1.3$
JSON schema valid-rate	raw	1.00	1.00	0.00
	fix	1.00	1.00	1.00
HumanEval pass@1 (trend only)	raw	0.079	0.043	0.000
	fix	0.079	0.085	0.079

a user observes, we measured two end-to-end quality metrics on *complete* generations from a single code model (Qwen2.5-Coder-7B), at the common settings $\theta \in \{1.0, 1.1, 1.3\}$, under the raw operator versus the fix (Table 4). On the JSON metric: prompting for a complete JSON object against one of six fixed schemas ($n = 48$ prompts) and validating with `json.loads` plus a type check, the raw penalty’s valid-rate falls from 1.00 at $\theta = 1$ to 0.00 at $\theta = 1.3$: at a routine **repetition_penalty=1.3** the operator yields no parseable, schema-valid JSON, while under the fix it stays at 1.00 across all θ . A HumanEval pass@1 run shows the same shape (raw 0.079 $\rightarrow$ 0.000; fix flat at ~ 0.079); we report only the *relative* trend there, because our lightweight completion-extraction harness depresses the absolute baseline far below the model’s true HumanEval, so the absolute numbers are a harness artifact and not a model fact. This is one model and a small probe, but it connects the per-token threshold (A2) to a user-facing failure. The exact θ at which a given model’s structured output collapses will itself vary with the model’s zero-point (Table 2); the mechanism is what generalizes, and the fix removes it.

The fix is a usable penalty, not a disabled one. A natural worry is that normalize-before-penalize removes the corruption only by weakening the penalty into a no-op. It does not: on the loop-prone gpt2 and pythia-2.8b, the normalized penalty still breaks degenerate loops, with the repetition rate falling monotonically in θ (gpt2 0.83 $\rightarrow$ 0.63, pythia-2.8b 0.80 $\rightarrow$ 0.60 over $\theta \in [1.0, 1.5]$). It is, as expected, *gentler per unit* θ than the raw operator, which reaches a repetition rate of ~ 0.01 –0.1 by $\theta = 1.3$: because the normalized penalty acts on bounded log-probabilities rather than gauge-dependent raw logits, the same θ is a smaller push, so the operative range shifts upward. This both confirms the fix retains anti-degeneration efficacy and quantifies the re-tuning caveat above.

6 Implications

Both effects replicate inside vLLM and llama.cpp. The results above are measured on the HuggingFace implementation; the same operator ships in vLLM and llama.cpp (Table 5), and we re-ran A1 and A2 inside each, through each stack’s own sampler and its own knob: vLLM 0.8.5.post1 (the operator in `vllm/main` at commit `e196268` is mathematically identical) and llama.cpp at master commit `4fc4ec55`. The A1 gauge probe passes its validity gate in both stacks (exactly 0 of 3,200 greedy tokens flip at $\theta = 1$) and reproduces the divergence at $\theta = 1.3$ on gpt2-large: flip rate 0.939 in vLLM and 0.941 in llama.cpp, against 0.941 on HuggingFace (Table 1). The A2 JSON task

(Qwen2.5-Coder-7B, same schemas and validator as Table 4) goes from 100% valid at $\theta \in \{1.0, 1.1\}$ to 16.7% at $\theta = 1.3$ in both stacks (HF: 0%; the residual valid outputs trace to bf16 numerics). llama.cpp’s default 64-token penalty window does not change the outcome: greedy JSON completes inside the window, and its output is byte-identical to whole-context penalization on all 144 paired generations.³ Scripts and raw outputs are in `code/smoke_*` and `results/smoke_*` alongside the main harness.

Penalty settings are not portable. Because the operator reads an unconstrained gauge (A1), a `repetition_penalty` value carries no fixed meaning across models or checkpoints. Default cookbooks, leaderboard configurations, and tuned presets that pin a single value implicitly assume a comparability the operator does not provide. Under the normalized operator of Section 5, the penalty’s magnitude is expressed in log-probability units and is at least dimensionally comparable across models, a precondition for any value to transfer.

Structured-output corruption is predictable. Practitioner folklore that repetition penalties “break JSON” or “mangle code” (reported anecdotally around vLLM and guided-decoding tooling, and a motivation for grammar-constrained decoding [23]) has, in A2, an exact account: the penalty flips a confident, correct delimiter to its runner-up whenever $g < z_{\text{top}}(1 - 1/\theta)$, and this reaches end-to-end output: at a routine `repetition_penalty=1.3` it takes schema-valid JSON output to 0% (Table 4). Practitioners who must keep a repetition penalty on for structured generation can either exempt grammar-obligatory tokens from the penalized set, or adopt the normalized operator, under which the sign-branch corruption disappears and JSON validity is restored to 100%. Related reports are current: 2026 issues against Gemma 4, in both the reference stack [16] and vLLM [19], document JSON generation collapsing into repetition that grammar-constrained decoding does not prevent (the repeated words are valid string content) and that `repeat_penalty` at 1.0, 1.15, and 1.5 does not fix. That failure mode is distinct from ours (there the penalty fails to help; here it causes the failure); we note the fielded mitigation is to detect and truncate the loop [20]. Either way, the multiplicative penalty is not a reliable instrument for structured generation.

Operational footprint. The two effects have different operational footprints, and the difference favors A1. A2’s deterministic corruption applies wherever an unconstrained, low-temperature multiplicative penalty meets structured output: grammar-constrained decoding [23] neutralizes it (the masked, grammar-invalid runner-up cannot be emitted), and raising temperature dilutes the deterministic flip into a merely probabilistic one. A1, by contrast, is *not* rescued by either: the gauge shifts the entire logit vector *before* temperature scaling and independently of any token mask, so it perturbs the whole distribution at any temperature and under any constraint set. The non-portability of penalty values (A1) therefore reaches every deployment of the multiplicative penalty, while the structured-output corruption (A2) is concentrated in the common but bounded regime of unconstrained greedy/low-temperature structured generation.

The general form. Any decoding-time operator that branches on, or thresholds against, an absolute (non-shift-invariant) function of the raw logit inherits a dependence on a coordinate the training objective leaves unconstrained. The CTRL sign-branch is the most widely deployed instance; the dividing line is absolute-zero-point dependence, not “reading magnitude” per se. Operators that

³Measuring inside llama.cpp also surfaced an unrelated defect: `-repeat-last-n -1` is documented as “context size” but the sampler clamps the value to 0, silently disabling the penalty (`src/llama-sampler.cpp`, `std::max(penalty_last_n, 0)`).

Table 5: Penalty form by stack, and exposure to the two effects. The multiplicative CTRL form (sign-branch on raw logits) is subject to A1 gauge non-invariance and the A2 threshold; the subtractive presence/frequency form subtracts a constant and is shift-invariant, hence immune to both. All three multiplicative stacks are measured directly (HuggingFace in the main experiments; vLLM and llama.cpp in the cross-stack replication, Section 6).

Stack / knob	Penalty form	Normalizes first?	A1?	A2?
HF <code>repetition_penalty</code>	multiplicative (CTRL)	no (default)	yes	yes
llama.cpp <code>repeat_penalty</code>	multiplicative (CTRL)	no	yes	yes
vLLM <code>repetition_penalty</code>	multiplicative (CTRL)	no	yes	yes
Hosted-API <code>presence_penalty</code>	subtractive (constant)	n/a	no	no
<code>frequency_penalty</code> (API/vLLM)	subtractive (count-scaled)	n/a	no	no

read only shift-invariant quantities are immune by construction: top- k , top- p /nucleus, min- p [13], typical sampling [12], and contrastive search [17] all act on the softmax distribution or on logit *differences*, where the gauge cancels.

7 Related work

Anti-degeneration decoding. Repetition penalties sit alongside truncated sampling [2, 5], unlikelihood training [22], contrastive decoding [18], and penalty redesigns such as the information-theoretic LZ penalty [4] as tools against degeneration. That literature asks *how much* to penalize, or proposes a better penalty; we ask, prior to that, whether the standard operator is well-defined, and show a fixed θ is not a fixed intervention.

Multiplicative vs. subtractive penalties (scope of A1). Two penalty families are deployed in practice, and the gauge dependence distinguishes them (Table 5). The *multiplicative* CTRL form (Eq. (1)) sign-branches on the raw logit and is the form shipped by HuggingFace and by llama.cpp’s `repeat_penalty` [3]; it is the one we study, and it is subject to A1. The *subtractive* presence/frequency penalties exposed by the major hosted APIs [14] (and also by vLLM [9]) instead subtract a constant (and, for the frequency variant, a count-scaled term) from the logit of seen tokens: $z_i \mapsto z_i - \alpha$. A constant subtraction commutes with the gauge ($(z + c\mathbf{1})_i - \alpha = (z_i - \alpha) + c$, and the c again cancels in softmax), so the subtractive family is *shift-invariant and immune to A1*; the count-scaled frequency term is likewise gauge-invariant since the counts do not depend on c . A2 has no analogue for the subtractive form at all, because there is no sign-branch. This is why we scope the title’s claim to the multiplicative operator: the ubiquity is real, but it is the ubiquity of *that* operator, not of every penalty knob.

Logit geometry. Work on glitch and under-trained tokens [10] exploits a model’s embedding geometry through specific anomalous tokens; our gauge dependence is instead a *global* additive freedom in the output logits, independent of any particular token.

Constrained decoding. Grammar-constrained decoding [23] routes around structured-output corruption by masking invalid tokens at the sampling layer; we instead characterize it in closed form and fix the operator itself. The two specific contributions (the gauge non-invariance of the sign-branched penalty and the closed-form delimiter threshold) are, to our knowledge, new.

8 Limitations

We study the specific HuggingFace/CTRL multiplicative operator of Eq. (1); the subtractive presence/frequency penalties (Section 7, Table 5) are shift-invariant and escape A1, and stacks that already normalize before penalizing are likewise exempt. The A1 flip-rate sweep spans five models up to 7B (three small base models plus Qwen2.5-7B and its instruction-tuned/RLHF variant, on which the effect is undiminished; Table 1), and the zero-point realism measurement (Table 2) covers all five checkpoints; we have not pushed beyond 7B or to the largest frontier models. The cross-stack replication (Section 6) uses vLLM 0.8.5.post1 (with the operator verified unchanged in `vllm/main` at `e196268`) and a single pinned llama.cpp master commit; its A1 leg covers gpt2-large only. A2 is demonstrated on two independently trained 7B code models (StarCoder2-7B and Qwen2.5-Coder-7B), which share the closed-form threshold and the gap-driven signature but differ in the magnitude of code-vs-prose concentration ($2.3\times$ versus $5.6\times$), so that secondary quantity should be read as model-dependent. Our A2 analysis isolates the “clean” positions where Eq. (4) is exact (top-1 seen and positive, runner-up unseen); positions where both the top choice and the runner-up are penalized obey a two-sided version of the inequality that we do not separately tabulate here. Finally, our core endpoints are mechanistic (per-token flips, threshold accuracy); we connect them to end-to-end quality on two tasks (Table 4), but a broader downstream evaluation (more models, more schemas, a canonical HumanEval harness for clean absolute pass@1, and prose-quality metrics) is left to future work.

9 Conclusion

The repetition penalty is treated as a clean, portable dial that leaves everything but repetition alone. As actually implemented (a sign-branched divide/multiply on raw logits), it is none of these. It is not gauge-invariant: a provably invisible shift of the logit zero-point, which already differs across real models, turns the same nominal θ into a different intervention, flipping half to nearly all greedy tokens at $\theta = 1.3$. And where it acts it is a closed-form corruption threshold, $g < z_{\text{top}}(1 - 1/\theta)$, that deterministically replaces a confident correct token with its runner-up, verified at balanced accuracy 0.999, with every flip landing on the runner-up. Both effects share one root cause and one remedy: normalize before penalizing. The operator that does this already ships in the library; it is off by default and runs after the penalty. The penalty is not the well-defined knob it is assumed to be, and making it one requires only a reorder of existing machinery.

Reproducibility

All experiments are inference-only and run on a single 24–48 GB GPU. The small A1 models run in fp32; the 7B models use bf16 (`run_a1.py -dtype bfloat16`) to fit the card, but the gauge shift and argmax decision are computed in fp32, so the no-op gate is exact regardless of model dtype. The pinned environment (`torch 2.6.0+cu124`, `transformers 5.11.0`), the public model revisions, and the exact scripts that produce every number and table are included with this paper: `run_a1.py/analyze_a1b.py` (A1, with `diag_divergence.py` for the divergence diagnostic) and `run_a2.py/analyze_a2.py` (A2, with `explore_delim.py` for the brackets-only re-slice). The fix demonstration (Table 3) is reproduced by `run_a1.py -fix / run_a2.py -fix` ($\rightarrow$ `results/a1b_fix/`, `results/a2_fix/`), and the second-model replication by `run_a2.py` on Qwen2.5-Coder-7B ($\rightarrow$ `results/a2_qwen/`). The end-to-end quality probe (Table 4) is produced by `run_a2_downstream.py/analyze_a2_downstream.py`, the fix loop-suppression check by `run_a1_loopcheck.py`,

and the zero-point realism measurement (Table 2) by `run_a1_zeropoint.py`. The cross-stack replication is produced by `code/smoke_vllm/` and `code/smoke_llamacpp/` (each with its own README, pinned environment, and build steps; raw outputs and per-stack reports in `results/smoke_*`). Pre-registered hypotheses and frozen decision rules, and a record of which sub-claims were verified, partial, or refuted, accompany the code.

A Pre-registered hypotheses: verified, partial, and refuted

Each experiment was pre-registered with a frozen decision rule before its confirmatory run; the rules and their raw verdicts ship with the code. The body reports the surviving claims; here we record the sub-hypotheses that were only partially confirmed or refuted, so the audit trail is complete without slowing the main argument.

A.1 A1: measurement channel and the shape of the curve

The effect is content divergence, not a repetition-rate change. The original hypothesis predicted that a weaker-gauge penalty would leave more repetition. At $\theta = 1.3$ that channel is invisible: the penalty breaks loops under *both* gauges, and a distinct-2 endpoint saturates at 1.000 for every c . The divergence instead manifests as different *content*, which is what the flip-rate endpoint measures and why we adopt it. In a non-saturated regime (the smallest $\theta = 1.02$) the repetition-rate channel is recoverable but weak: for gpt2-large and pythia-2.8b the repetition rate is ordered in the predicted direction across c (e.g. gpt2-large $0.823 \rightarrow 0.771$ from $c = -5$ to $c = +5$), while for gpt2 it is flat and noisy. The strong, unambiguous signal is content divergence.

Divergence versus θ : off-zero leap, model-size-dependent shape, and a monotonicity clause that overreached. The flip rate rises from exactly 0 at $\theta = 1$ to a large value as soon as the penalty turns on (0.61–0.78 at $\theta = 1.02$, near-ties tipped by the sign-branch), with per- θ bootstrap 95% CIs far above zero at every $\theta > 1$ (percentile bootstrap over the 16 prompts, 10^4 resamples). The later shape varies with model size: the larger models climb monotonically toward ~ 0.9 , while small gpt2 peaks near $\theta = 1.05$ and trends down (as strong loop-breaking forces both gauges into similar non-repetitive text). Our pre-registered A1 rule included a clause requiring the flip rate to be *monotone non-decreasing* in θ ; that fails for gpt2 and pythia-2.8b, so the frozen rule returned “not confirmed” for those two. We leave the frozen verdict as printed, but the clause was an overreach: nothing in the claim (“a gauge no-op changes behavior”) entails that the *amount* of divergence grow monotonically with θ . The substantive clauses (flip rate jumps off zero; bootstrap CI excludes zero) hold on all three models. We also report the gpt2 non-monotonicity as directional only: at $n = 16$ the bump and its $\theta = 1.3$ value have overlapping CIs.

A.2 A2: two refuted embellishments and an uninformative control

“Corrupts the most confident tokens first”: refuted. The reasoning was that the vulnerable gap $z_{\text{top}}(1 - 1/\theta)$ grows with z_{top} , so high-confidence tokens should be hit first. It is incomplete: confident tokens also tend to have *larger* gaps g , and the gap term dominates. Flipped tokens have *lower* mean z_{top} than non-flipped tokens: 17.5 versus 22.1 over all penalized positions (difference 95% CI $[-4.7, -4.4]$), and 19.9 versus 23.9 restricted to delimiters (CI $[-4.4, -3.6]$); the same ordering holds on Qwen2.5-Coder (23.9 versus 28.8, CI $[-5.1, -4.8]$). Corruption is *gap-driven*: it lands on tokens the model is *less* sure about. The threshold formula stands; the prediction about *which* tokens it selects was wrong.

Code-specificity is real but model-dependent. The structural-token flip rate per generated token is higher in code than prose, but the ratio varies with the model: on StarCoder2 it is 0.138 versus 0.060, only $2.3\times$ ($2.4\times$ on a brackets-only re-slice), below the pre-registered $\geq 5\times$; on Qwen2.5-Coder it is 0.196 versus 0.035, a $5.6\times$ that clears the bar. Two things are stable. First, prose is corrupted at a comparable *overall* rate (on StarCoder2 prose flips *more* tokens than code, 36% versus 31%), so the penalty corrupts all token types at a similar rate and is not syntax-specific. Second, code concentrates *which* tokens are hit: 19% of code’s flips land on delimiters versus 7% in prose. The concentration on *structural* tokens is because structured output exposes more grammar-obligatory delimiters to a gap-driven threshold.

The validity “cliff” was uninformative as measured. We had intended to show a sharp drop in parse validity as θ crosses the threshold, but StarCoder2’s free continuations are truncated fragments, so `json.loads` sits at the floor (0.00) even at $\theta = 1.0$ and bracket-balance error versus θ is noise. We make no cliff claim from it; it was pre-registered as descriptive and non-gating. (The clean end-to-end measurement on complete documents is in Section 5, Table 4.)

References

- [1] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning (ICML)*, 2023.
- [2] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- [3] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023. The `repeat_penalty` sampler applies the CTRL divide/multiply form to raw logits.
- [4] Antonio A. Ginart, Naveen Kodali, Jason Lee, Caiming Xiong, Silvio Savarese, and John R. Emmons. LZ penalty: An information-theoretic repetition penalty for autoregressive language models. *arXiv preprint arXiv:2504.20131*, 2025.
- [5] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.
- [6] HuggingFace Transformers contributors. Fix decoding score comparison when using logits processors or warpers. <https://github.com/huggingface/transformers/pull/10638>, 2022. Merged. Logits processors can leave beam scores un-normalized; the fix re-normalizes (via `log_softmax`) so beam comparison is fair.
- [7] Binyuan Hui, Jian Yang, Zeyu Cui, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- [8] Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. CTRL: A conditional transformer language model for controllable generation. *arXiv preprint arXiv:1909.05858*, 2019.

- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [10] Sander Land and Max Bartolo. Fishing for Magikarp: Automatically detecting under-trained tokens in large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [11] Anton Lozhkov, Raymond Li, Loubna Ben Allal, et al. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19737*, 2024.
- [12] Clara Meister, Tiago Pimentel, Gian Wiher, and Ryan Cotterell. Locally typical sampling. *Transactions of the Association for Computational Linguistics (TACL)*, 11, 2023.
- [13] Minh Nguyen, Andrew Baker, Clement Neo, Allen Roush, Andreas Kirsch, and Ravid Shwartz-Ziv. Turning up the heat: Min-p sampling for creative and coherent LLM outputs. *arXiv preprint arXiv:2407.01082*, 2024.
- [14] OpenAI. OpenAI API reference: `presence_penalty` and `frequency_penalty`. <https://platform.openai.com/docs/api-reference>, 2024. Subtractive penalties: a constant (and count-scaled) term subtracted from the logits of seen tokens.
- [15] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [16] rnh0 and Gemma contributors. [BUG] Gemma 4 token repetition collapse during long generation. <https://github.com/google-deepmind/gemma/issues/622>, 2026. JSON generation collapses into repetition; grammar-constrained decoding cannot prevent it (repeated words are valid string content) and `repeat_penalty` at 1.0/1.15/1.5 has no effect.
- [17] Yixuan Su and Nigel Collier. Contrastive search is what you need for neural text generation. *Transactions on Machine Learning Research (TMLR)*, 2023.
- [18] Yixuan Su, Tian Lan, Yan Wang, Dani Yogatama, Lingpeng Kong, and Nigel Collier. A contrastive framework for neural text generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [19] vLLM contributors. [Bug] Gemma 4 generates infinite repetition loops, especially with structured output (json schema). <https://github.com/vllm-project/vllm/issues/40080>, 2026.
- [20] vLLM contributors. [Bugfix] auto-enable repetition detection for grammar-constrained structured output. <https://github.com/vllm-project/vllm/pull/40099>, 2026. Mitigates the loop by detecting and truncating repetition rather than changing the penalty.
- [21] von Platen, Patrick and HuggingFace Transformers contributors. Repetition penalty works falsely in case the logit of the token is negative. <https://github.com/huggingface/transformers/issues/2302>, 2019. GitHub issue reporting that dividing a negative logit by a penalty > 1 raises its probability; the sign-branched divide/multiply form is the community fix.

- [22] Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training. In *International Conference on Learning Representations (ICLR)*, 2020.
- [23] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
- [24] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.